

CompSci 330: Design and Analysis of Algorithms

Summer 2024, Homework 5

Due: Mon 8/5 @ 6pm or Fri 8/9 @ 11:59pm

Directions

How to Do Homework. We recommend the following three step process for homework to help you learn and prepare for exams.

1. Give yourself 15-20 minutes per problem to try to solve on your own, without help or external materials, as if you were taking an exam. Try to brainstorm and sketch the algorithm for applied problems. Don't try to type anything yet.
2. After a break, review your answers. Lookup resources or get help (from peers, office hours, Ed discussion, etc.) about problems you weren't sure about.
3. Rework the problems, fill in the details, and typeset your final solutions.

Submission. Your written solutions should be typeset and submitted as a single PDF on Gradescope. $\text{\LaTeX}$ ¹ is preferred but not required. If you use another editor for your solutions (such as Microsoft Word), you should convert the final document to a PDF, confirm that the symbolic math from the equation editor is properly formatted, and submit the PDF. You must mark the locations of your solutions to individual problems on Gradescope as explained in [the documentation](#). Implementation problems will request that you submit code separately on Gradescope to be autograded.

Writing Expectations. If you are asked to provide an algorithm, you should clearly and unambiguously define every step of the procedure as a combination of precise sentences in plain English or pseudocode. If you are asked to explain your algorithm, its runtime complexity, or argue for its correctness, your written answers should be clear, concise, and should show your work. Do not skip details but do not write paragraphs where a sentence suffices. Results covered in lecture may be referenced and used directly without any justification of their correctness (unless asked for).

Collaboration. You can (and should!) work with a partner, in which case you will submit a single solution as a group on Gradescope *by adding your partner to your submission on Gradescope* ([see here for directions](#)). *Always cite anyone/anything consulted* outside of course materials, including a brief description of their contribution to your work. See also our policy on collaboration on the [course webpage](#).

Grading. Theory problems will be graded on an S/U scale (for each sub-problem). Applied problems typically have a separate autograder where you can see your score. **The lowest scoring problem is dropped.** See the [course assignments webpage](#) for more grading details.

¹If you are new to $\text{\LaTeX}$, you can download it for free at [latex-project.org](https://www.latex-project.org) or you can use the popular and free (for a personal account) cloud-editor [overleaf.com](https://www.overleaf.com). We also recommend [overleaf.com/learn](https://www.overleaf.com/learn) for tutorials and reference.

Comments for HW5:

- Problems 1 and 2 have **optional deadlines on Monday @ 6pm**. Feedback will be provided before the review on Tuesday, along with our solutions for those problems. Of course, the solutions should not be shared with other groups before the final deadline on Friday.
- For Problems 1 and 2, one should carefully describe a reduction to max-flow (or problems we reduced to max-flow in lecture, such as edge-/vertex-disjoint paths or maximum matchings). Your solution should involve the following clearly defined:
 1. The vertices and edges of the graph.
 2. If reducing to max-flow, the capacities of the edges.
 3. The specific problem that you are reducing the given problem to. Write it in terms of the graph, e.g. “compute the max-flow from s to vertex t in the constructed graph.”
 4. How to solve that graph problem, and how to transform its solution back to a solution for the given problem.
 5. Finally, a runtime analysis in terms of the given input (**NOT** just the number of vertices and edges of the graph you constructed).
- There is no applied problem for this HW.

Problem 1

The Trinity Curriculum Development Committee (TCDC) recently submitted a new curriculum, which was approved by the Arts & Sciences Council. The new curriculum requires a student to take a number of classes r_i from m different categories G_i in order to graduate. Let n be the number of courses.

For example, suppose there are $n = 5$ courses A, B, C, D, E and $m = 2$ categories:

- You must take at least $r_1 = 2$ courses from the subset $G_1 = \{A, B, C\}$.
- You must take at least $r_2 = 2$ courses from the subset $G_2 = \{C, D, E\}$.

Then a student who has only taken courses B, C, D cannot graduate, but a student who has taken either A, B, C, D or B, C, D, E can graduate.

Describe and analyze an algorithm to determine whether a given student can graduate. The input to your algorithm is the list of the m categories (each specifying a subset G_i of the n courses and the number of courses r_i that must be taken from that subset) and the list of courses that the student has taken. Justify the correctness of your reduction and analyze the runtime.

Problem 2

Suppose we are given an array $A[1..m][1..n]$ of non-negative numbers. We want to *round* A to an integer matrix, by replacing each entry x in A with either $\lfloor x \rfloor$ or $\lceil x \rceil$, without changing the sum of entries in any row or column of A . For example:

$$\begin{bmatrix} 1.2 & 3.4 & 2.4 \\ 3.9 & 4.0 & 2.1 \\ 7.9 & 1.6 & 0.5 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 4 & 2 \\ 4 & 4 & 2 \\ 8 & 1 & 1 \end{bmatrix}$$

- a. Describe and analyze an efficient reduction-based algorithm that either rounds A in this fashion, or reports correctly that no such rounding is possible. Justify that your algorithm is correct, and analyze the runtime. [Hint: There is a correct reduction involving a flow network with one vertex per row and one vertex per column.]
- b. Justify that a matrix can be rounded if and only if the sum of entries of each column is an integer and the sum of entries of each row is an integer. [Hint: Consider a particular **fractional** flow on your graph. Recall properties of flow networks with integer capacities.]

Comments for Problems 3-4:

In Thursday's lecture we defined the complexity classes P , NP , $co-NP$, and defined NP-hardness. To show a problem Π is NP-hard, give a polynomial-time reduction from a known NP-hard problem Ξ to Π ; that is, describe a polynomial-time algorithm for Ξ assuming there exists a polynomial-time algorithm for Π .

For full credit, any reduction must include a justification that the algorithm is correct (for all instances of Ξ). For example, on Monday, our reduction from 3SAT to MAXINDSET includes arguing its polynomial runtime and that a 3-CNF Boolean formula Φ of k clauses is satisfiable **if and only if** our constructed graph G contains a maximum independent set of size k , which implies our overall algorithm, and hence the reduction, is correct.

You may use the fact that any NP-hard problem discussed in lecture/discussion is indeed known to be NP-hard, e.g., CIRCUITSAT, SAT, 3SAT, MAXINDSET, VERTEXCOVER, CLIQUE, DIRHAMCYCLE, DIRHAMPATH. Some of these will be defined on Monday.

Problem 3

A *Hamiltonian cycle* is a (simple) cycle that visits all vertices in the graph exactly once. In Monday's lecture, we will see that DIRHAMCYCLE, detecting whether a given directed graph has a Hamiltonian cycle, is NP-hard. The UNDIRHAMCYCLE problem is the same except that the given graph and cycle to detect are undirected.

Describe a polynomial-time reduction from DIRHAMCYCLE to UNDIRHAMCYCLE, and justify its correctness in order to conclude that UNDIRHAMCYCLE is NP-hard. [Hint: Consider replacing each node u with multiple copies of u . How many copies should be made, how should the copies of one node be connected via edges, and how should the copies of different nodes be connected based on the original edges?]

Problem 4

In this problem we prove that, as for many NP-hard problems, allowing constant additive error does not make the problem any "easier." We define the ALMOSTMAXINDSET problem as follows: Given an undirected graph $G = (V, E)$, output an integer k in the range $|S^*| - 330 \leq k \leq |S^*|$, where $S^* \subseteq V$ is a maximum independent set in G . Prove that ALMOSTMAXINDSET is NP-hard. Here are some hints:

- There is nothing special about the constant 330. Assume it is, say, 3 instead when thinking about the problem.
- Reduce from MAXINDSET and modify the graph in such a way that the size of the maximum independent set in the resulting graph increases in a controlled, predictable way.
- An **incorrect** reduction is to simply add many isolated vertices (e.g., ~ 330) to the graph. Why is this incorrect? What can you add a sufficiently large amount of instead to obtain a correct reduction?

- *The size of any independent set is an integer.*